

1 The problem

A wallet's users do three things: send money, ask for it, split a bill. Underneath, one rule has to hold no matter what: **the money in the system never changes.**

Not "mostly". Never. Not under a retry, not under twenty simultaneous transfers from one account, not when a split does not divide evenly, not when a request to save and export your state comes back twice.

Getting this right by reading diffs is the expensive part. A senior reviewer takes about two hours per change to a money-moving path, and they are reviewing *correctness under concurrency*, which is exactly the thing a diff does not show.

2 What we built

A factory, not a service. Three coding agents that build the service:

- one **implements** to the specification, runs the checks, commits
- one **verifies** that exact commit and **veto**es it if it is wrong

They share one room. Once a stage is dispatched, no human is in the loop until the stage is locked — and that is checkable rather than promised: all 121 messages in the exported room carry an agent sender, and not one is from a human.

3 Why a factory and not a better code review

Code review catches what a human thought to look at. The factory's reviewer was told what to look for: same key fired ten times at once, a response lost after the server already committed, a snapshot restored twice, a page whose state went stale under it.

Those are not things a reviewer remembers at 5pm on a Friday. They are a list.

4 The reviewer finds what the shipped checks never asked for

The reviewer never writes service code. It cannot make a failing check pass by editing the thing being checked.

It writes its own scenarios, in notes the implementer is forbidden to read. What it found that the published checks never asked for:

	SCENARIO	WHAT THE SHIPPED CHECKS WOULD HAVE SAID	STAGE LOCKED AT
stage 1	a snapshot restored twice duplicates records	fine	35e2fa1
stage 2	a hidden error element survives a successful payment	fine	4004384
stage 2	a list reload wipes the error the failed action just set	fine	4004384
stage 2	the split preview disagrees with the server by one minor unit	fine	4004384
stage 4	the refund cap used the original amount, so a correction downward did not tighten it	fine	b88cb7b
stage 4	a batch correction could be applied to a settlement without all of its members	fine	b88cb7b

Six. Slide 5 says what the room did about them, and what it did not do.

5 What the run admits

Each one of those six is a commit whose subject says what changed — `4004384` names three of them in a single line. Each was reported with numbers, and the fixes came back through the room as new commits.

But they came back as **acceptances with findings attached, not as vetoes**. In this run the reviewer never rejected anything: every verdict the room carries is an ACCEPT, each naming the revision it was issued against. The veto the factory is built around is still untested in this room, and that is stated in FACTORY.md §4 rather than left for a judge to discover.

Stage 3 is the honest gap. Its eleven scenarios ran and all of them passed, so there is nothing to list. Two of the four stages' holdout files were also written *after* the verdict rather than before it, which is backwards; those files say so at the top, and the fix — a reviewer that writes its scenario first and hands it over as the task — is slide 11.

6 What it produced

Four stages of the wallet specification. Each folder is a complete service that builds from a clean container and passes every earlier stage's checks as well as its own.

STAGE	SHIPPED CHECKS RUN AGAINST IT
1 — the API	147 / 147
2 — the browser product	147 + 35
3 — history and corrections	147 + 35 + 6
4 — refunds and batch corrections	147 + 35 + 6 + 5

7 Verified by the event's own harness

Not by us. The organiser's harness, isolated mode, building each folder from scratch:

```
stage-1/  claims stage 1      share 1.0
stage-2/  claims stage 2      share 1.0
stage-3/  claims stage 3      share 1.0
stage-4/  claims stage 4      share 1.0
highest contiguous stage: 4
```

Nothing is claimed on our word. The report is committed at [docs/harness-runs/](#).

And the honest caveat: the event ships part of each stage's checks. Stage 1's 147 and stage 2's 35 are a real sample of the grading suite; stage 3's 6 and stage 4's 5 are close to a smoke test. What carries those two is the reviewer's own adversarial work in [holdouts/](#), not the six and five. That number is directional, and we say so in the factory document rather than let a judge work it out.

8 What it cost

\$0. Every seat ran on a free tier.

Room window: about 28 hours wall-clock across two calendar days, of which the useful work was roughly four. The rest was infrastructure: restarting seats, draining a backlog from an abandoned earlier room, and one long verification that had to be split in two.

9 It failed in instructive ways

A factory that never fails is a diagram.

stranding a later commit. Recovered from the reflog; the dispatch for the next stage now forbids a bare `git checkout`.

the room proved nothing. It was the script.

blind, partitioned ownership, and had it reconciled.

All three are written up in `FACTORY.md`, because that is what saves the next team the day.

10 Who it is for

Teams who ship money-moving code and cannot afford a two-hour review per change — and who do not trust a black box to do it.

The factory is not vendor-specific. Three mandates, three adapters, one room. Point it at a different specification and the routing, the veto, and the discipline transfer unchanged.

11 What we would do next

implementation rather than after, would catch more.

factory currently solves with a ruling instead of a lock.

instead of discovering the gap afterwards. That is the real version of test-first, and it needs a seat that can hold a spec without writing code.